DEVOPS FINAL PROJECT

Design, Implementation and Documentation of a DevOps Infrastructure

Project	Restaurant Web Application
Student	Hasmik Levonyan
GitHub repository	https://github.com/HasmikL/Devops_2026
Application project	restaurant-devops
Operating system	Rocky Linux
Virtualization platform	Oracle VirtualBox
Main technologies	Git, GitLab, Jenkins, Docker, Docker Registry, Ansible, Trivy, OWASP ZAP, HAProxy, Keepalived, PostgreSQL, Prometheus and Grafana
Project type	Practical DevOps Infrastructure and CI/CD Implementation
Academic year	2026

Table of Contents

1. Introduction 6

2. Project Objectives 7

3. Scope of the Project 8

3.1 Laboratory Implementation 8

3.2 Target Production-Like Architecture 8

4. Technologies and Their Responsibilities 9

5. Expected Result 10

6. Virtual Machine and Network Configuration 11

6.1. Virtualization Environment 11

6.2. Network Architecture 11

6.3. Verify Network Configuration 12

6.4. Verify the Operating System 12

6.5. Check Available System Resources 12

6.6. Test Network Connectivity 12

6.7. Required Network Ports 13

6.8. Check Listening Ports and Service Status 13

6.9. Security Considerations 13

6.10. Verification Checklist 14

7. GitLab, Git Workflow, and Docker Registry 15

7.1. Source Code Management 15

7.2. Verify Git Installation 15

7.3. Clone the Application Repository 15

7.4. Git Development Workflow 15

7.5. Common Git Commands 16

7.6. Docker Registry 16

7.7. Verify Docker Installation 17

7.8. Build a Docker Image 17

7.9. Tag the Image for the Registry 17

7.10. Publish the Image 17

7.11. Verify Registry Connectivity 17

7.12. Pull the Image on the Deployment Server 18

7.13. Registry and Image Verification Checklist 18

7.14. Expected Result . . . 18

8. Jenkins CI/CD Pipeline . . . 19

8.1. Introduction to Jenkins . . . 19
8.2. Continuous Integration and Continuous Deployment . . . 19
8.3. Jenkins Architecture . . . 20
8.4. Verify Jenkins Availability . . . 20
8.5. Verify the Jenkins Job . . . 20
8.6. Jenkinsfile . . . 21
8.7. Jenkins Credentials and Permissions . . . 22
8.8. Execute the Pipeline . . . 23
8.9. Verify the Pipeline Result . . . 23
8.10. Troubleshooting . . . 23
8.11. Verification Checklist . . . 24
8.12. Expected Result . . . 25

9. Docker Deployment and Ansible Automation . . . 26

9.1. Introduction to Ansible . . . 26
9.2. Ansible Architecture . . . 26
9.3. Verify Ansible Installation . . . 26
9.4. Inventory Configuration . . . 27
9.5. Test SSH Connectivity . . . 27
9.6. Docker Deployment Playbook . . . 27
9.7. Execute the Deployment . . . 28
9.8. Verify the Docker Container . . . 29
9.9. Deployment Idempotency . . . 29
9.10. Rollback Strategy . . . 29
9.11. Troubleshooting . . . 30
9.12. Verification Checklist . . . 30
9.13. Expected Result . . . 31

10. Security Scanning with Trivy and OWASP ZAP . . . 32

10.1. Introduction . . . 32
10.2. Trivy - Container Image Vulnerability Scanning . . . 32
10.3. OWASP ZAP - Dynamic Application Security Testing . . . 33
10.4. Integrating Security Scanning into Jenkins . . . 35
10.5. Troubleshooting . . . 35
10.6. Security Testing Checklist . . . 36
10.7. Expected Result . . . 36

11. High Availability and Load Balancing with HAProxy and Keepalived 37

11.1. Introduction . . . 37
11.2. Target Architecture . . . 37
11.3. Prerequisites . . . 37
11.4. Install HAProxy . . . 38
11.5. Configure HAProxy . . . 38
11.6. Install Keepalived . . . 39
11.7. Configure Keepalived . . . 39
11.8. Start and Verify Keepalived . . . 40
11.9. High-Availability Tests . . . 41
11.10. Troubleshooting . . . 42
11.11. Limitations and Considerations . . . 42
11.12. Completion Checklist . . . 42
11.13. Expected Result . . . 43

12. PostgreSQL Replication, Backup, and Recovery 44

12.1. Introduction . . . 44
12.2. Target Architecture . . . 44
12.3. Verify PostgreSQL . . . 44
12.4. Configure Streaming Replication . . . 45
12.5. Verify Replication . . . 47
12.6. Create a Logical Database Backup . . . 47
12.7. Restore a Database Backup . . . 48
12.8. Backup Retention and Storage . . . 48
12.9. Recovery and Failover Considerations . . . 49
12.10. Troubleshooting . . . 49
12.11. Completion Checklist . . . 49
12.12. Expected Result . . . 50

13. Monitoring and Alerting with Prometheus and Grafana 51

13.1. Introduction . . . 51
13.2. Target Architecture . . . 51
13.3. Monitoring Requirements . . . 51
13.4. Install Prometheus and Node Exporter . . . 52
13.5. Configure Docker Compose . . . 52
13.6. Verify Prometheus . . . 53
13.7. Configure Grafana . . . 53
13.8. Configure a Basic Alert Rule . . . 54
13.9. Monitoring Best Practices . . . 55

13.10. Troubleshooting 55
13.11. Completion Checklist 56
13.12. Expected Result 56

14. Testing, Rollback, Troubleshooting, and Final Conclusions 57

14.1. Introduction 57
14.2. End-to-End DevOps Workflow 57
14.3. Pre-Deployment Checklist 58
14.4. Verify the Application Deployment 58
14.5. Rollback Procedure 59
14.6. Failure Testing 60
14.7. Troubleshooting Reference 61
14.8. Evidence Collection 61
14.9. Final Project Assessment 62
14.10. Limitations and Future Improvements 62
14.11. Conclusion 63

1. Introduction

Modern software development requires more than writing application code. Applications must be built, tested, deployed, monitored, secured and recovered when failures occur.

DevOps combines development practices, operations, automation and monitoring to make the software delivery process more reliable and repeatable.

The purpose of this final project is to design, implement and document a DevOps infrastructure for a Restaurant web application. The project uses a virtualized Linux environment to demonstrate source-code management, continuous integration, containerization, automated deployment and security testing.

The laboratory environment combines Git/GitLab, Jenkins, Docker, a private Docker Registry, Ansible, Trivy and OWASP ZAP. These tools provide the foundation for automating the application lifecycle from a source-code change to a tested container deployment.

The target architecture extends the laboratory environment with load balancing, high availability, database replication, backup and recovery, monitoring, alerting and rollback mechanisms.

This document describes the purpose of each component, the configuration files, the commands required to configure and verify the services, and the expected behavior of the infrastructure.

A distinction is maintained between components demonstrated in the laboratory and components that remain planned for the complete production-like architecture. This ensures that the documentation accurately represents the implementation status.

2. Project Objectives

The primary objective is to demonstrate how a web application can be managed through an automated DevOps workflow.

The specific objectives are:

1. Create and manage a version-controlled source-code repository.
2. Configure Linux servers for application and infrastructure services.
3. Containerize the application using Docker.
4. Build and version Docker images.
5. Store images in a private Docker Registry.
6. Configure Jenkins to execute a CI/CD pipeline.
7. Execute automated tests before deployment.
8. Integrate Trivy for container vulnerability scanning.
9. Use OWASP ZAP for dynamic web application security testing.
10. Automate server configuration and deployment using Ansible.
11. Verify application availability using health checks.
12. Design load balancing across multiple application servers.
13. Design load-balancer high availability using Keepalived and a Virtual IP.
14. Design a PostgreSQL database layer with backup and recovery.
15. Design infrastructure and application monitoring.
16. Configure alerting for important failures.
17. Define rollback and recovery procedures.
18. Test failure scenarios and document the results.
19. Maintain complete technical documentation of the infrastructure.

3. Scope of the Project

The project has two related scopes.

3.1 Laboratory Implementation

The laboratory implementation focuses on the services used to build and test the application delivery workflow:

- Linux virtual machines.
- Git and GitLab.
- Jenkins.
- Docker.
- Docker Registry.
- Ansible inventory and playbooks.
- Trivy vulnerability scanning.
- OWASP ZAP dynamic security testing.
- Application container deployment.
- Troubleshooting and service verification.

3.2 Target Production-Like Architecture

The target architecture extends the laboratory with the following components:

- Two application servers.
- Two load balancers.
- HAProxy traffic distribution.
- Keepalived VIP failover.
- PostgreSQL replication and failover.
- Automated database backups.
- Restore testing.
- Prometheus metrics collection.
- Grafana dashboards.
- Alerting.
- Automated deployment rollback.

These components are considered implemented only after their configuration and operation have been verified.

4. Technologies and Their Responsibilities

Technology	Responsibility
Rocky Linux	Provides the Linux server environment.
VirtualBox	Runs isolated virtual machines for the laboratory.
Git	Tracks source-code changes and versions.
GitLab	Hosts the application repository used by the CI/CD workflow.
Jenkins	Automates builds, tests, security checks and deployment stages.
Docker	Packages the application into a container image.
Docker Registry	Stores versioned container images.
Ansible	Automates server configuration and application deployment.
Trivy	Scans container images and dependencies for known vulnerabilities.
OWASP ZAP	Tests a running web application for potential security issues.
HAProxy	Distributes application traffic across backend servers.
Keepalived	Provides VIP failover between load balancers.
PostgreSQL	Provides relational database storage.
Prometheus	Collects infrastructure and application metrics.
Grafana	Displays metrics in dashboards.

5. Expected Result

The expected result is a documented DevOps workflow in which application changes are managed through source control, tested by Jenkins, packaged into Docker images, stored in a registry and deployed using automation.

The target workflow is:

```
Developer -> Git Repository -> Jenkins -> Automated Tests -> Security Scanning
          -> Docker Build -> Registry -> Ansible Deployment -> Health Check -> Application
```

The supporting infrastructure provides the design for load balancing, high availability, database protection, monitoring, alerting and recovery.

The final demonstration should include both a successful deployment and selected failure scenarios, with evidence showing how the system responds and how service is restored.

6. Virtual Machine and Network Configuration

6.1. Virtualization Environment

The project is developed in a virtualized environment using Oracle VirtualBox and Rocky Linux.

Virtualization allows multiple independent servers to run on a single physical computer. Each virtual machine can host a specific DevOps component, such as Jenkins, GitLab, Docker, or a database.

The main technologies used in the environment are:

- **Host operating system:** macOS
- **Virtualization software:** Oracle VirtualBox
- **Server operating system:** Rocky Linux
- **Container platform:** Docker
- **Source code management:** Git and GitLab
- **CI/CD automation:** Jenkins
- **Configuration management:** Ansible
- **Security testing:** Trivy and OWASP ZAP

This architecture provides an isolated environment for developing, testing, deploying, monitoring, and recovering the Restaurant web application.

6.2. Network Architecture

The virtual machines communicate through a virtual network configured in VirtualBox. Each server must have a valid IP address and appropriate network connectivity to communicate with the other components.

The following IP addresses and ports were used in the previous laboratory environment. They must be verified against the current virtual machines before being treated as the final configuration.

Component	Previously used address	Purpose
GitLab / Docker Registry	`192.168.20.159`	Source code management and container image storage
Jenkins	`192.168.20.162:8081`	CI/CD pipeline execution
Restaurant application	`3001:3000`	Application port mapping
PostgreSQL / Nginx WAF	`192.168.20.37`	Database and web security components
Restaurant application / Registry host	`192.168.20.39`	Application-related services

Important: These are historical laboratory values, not a verified description of the current environment. Update the table if the IP addresses, port mappings, or service locations have changed.

6.3. Verify Network Configuration

Run the following commands on each Rocky Linux virtual machine:

```
hostnamectl
ip -br address
ip route
```

These commands provide the following information:

- `hostnamectl` displays the hostname and operating system information.
- `ip -br address` displays network interfaces and their IP addresses in a compact format.
- `ip route` displays the routing table, including the default gateway.

The results help confirm the identity and network configuration of each server.

6.4. Verify the Operating System

Run:

```
cat /etc/os-release
uname -r
uname -m
```

These commands display:

- The Linux distribution and version.
- The currently running Linux kernel version.
- The CPU architecture, such as `x86_64` or `aarch64`.

Architecture verification is important because software packages and container images may support only specific processor architectures.

6.5. Check Available System Resources

DevOps tools can consume considerable CPU, memory, and disk space. Jenkins builds, Docker images, and OWASP ZAP security scans may require additional resources.

Run:

```
free -h
df -h
lsblk
nproc
```

Command descriptions:

- `free -h` displays memory and swap usage.
- `df -h` displays filesystem capacity and available disk space.
- `lsblk` lists block devices and partitions.
- `nproc` displays the number of available processing units.

Checking disk space is especially important because a full filesystem can interrupt Jenkins builds, Docker image downloads, and security scans.

6.6. Test Network Connectivity

To test connectivity between servers, use:

```
ping -c 4 192.168.20.159
ping -c 4 192.168.20.162
```

Replace these example addresses with the actual IP addresses of the virtual machines.

If `ping` fails, it does not necessarily mean that the server is unavailable. ICMP traffic may be blocked by a firewall.

To test an HTTP service, use:

```
curl -I --connect-timeout 5 http://192.168.20.162:8081
```

If Jenkins is accessible at that address and port, the command should return an HTTP response. If it fails, check the IP address, service status, firewall rules, and listening port.

6.7. Required Network Ports

The following table lists common ports associated with the project. Actual port usage depends on the deployed configuration.

Service	Port	Purpose
SSH	22/TCP	Remote server administration
HTTP	80/TCP	Web traffic
HTTPS	443/TCP	Encrypted web traffic
Jenkins	8081/TCP	Jenkins web interface in the laboratory
Docker Registry	5000/TCP	Container image push and pull
Restaurant application	3001/TCP	Example host port mapped to container port 3000
PostgreSQL	5432/TCP	Database connections
Prometheus	9090/TCP	Metrics collection and monitoring interface
Grafana	3000/TCP	Monitoring dashboards
HAProxy statistics	Configurable	Load-balancer statistics interface

The presence of a port in this table does not mean it is currently open. Only expose the ports required by the actual deployment.

6.8. Check Listening Ports and Service Status

To list listening TCP and UDP ports:

```
sudo ss -lntup
```

To identify failed system services:

```
systemctl --failed
```

To check individual services:

```
sudo systemctl status sshd
sudo systemctl status docker
sudo systemctl status jenkins
```

Run each service check on the server where that service is installed. A service that is not installed may return a message indicating that its unit could not be found.

6.9. Security Considerations

The following security principles should be applied when configuring the virtual infrastructure:

1. Restrict SSH access to authorized users and networks.

2. Open only the network ports required by the project.
3. Restrict PostgreSQL access to authorized application servers.
4. Protect the Jenkins, Grafana, and Docker Registry interfaces from unauthorized access.
5. Never commit passwords, access tokens, private keys, or other secrets to the Git repository.
6. Use HTTPS and TLS for production-facing services.
7. Back up existing configuration files before making significant changes.
8. Review firewall rules and service permissions regularly.

6.10. Verification Checklist

Before proceeding to the next stage, verify the following items:

- ☐ All required virtual machines are running.
- ☐ Each server has the expected hostname and IP address.
- ☐ The operating system and CPU architecture have been identified.
- ☐ Sufficient memory and disk space are available.
- ☐ Required servers can communicate over the network.
- ☐ Required services are listening on their expected ports.
- ☐ Firewall rules permit necessary traffic without exposing unnecessary services.
- ☐ The network diagram and IP address table match the actual environment.

The purpose of this stage is to establish a verified foundation for the DevOps pipeline. Incorrect IP addresses, unavailable ports, or insufficient system resources can prevent later stages, including image transfer, automated deployment, security testing, and monitoring, from working correctly.

7. GitLab, Git Workflow, and Docker Registry

7.1. Source Code Management

Git is a distributed version control system used to track changes in source code and configuration files. GitLab provides a centralized platform for storing repositories, reviewing changes, and integrating source code with CI/CD tools.

In this project, Git and GitLab support the Restaurant application's development and provide Jenkins with the source code required to execute the deployment pipeline.

The project documentation repository is:

`https://github.com/HasmikL/Devops_2026`

The Restaurant application repository used in the laboratory was named `restaurant-devops`. These are separate repository references and should not be confused with each other.

7.2. Verify Git Installation

Run the following commands on the development machine or server where Git is installed:

```
git --version
git config --global --list
```

The first command displays the installed Git version. The second displays the configured Git identity and other global settings.

Configure the Git author identity if it has not already been configured:

```
git config --global user.name "HasmikL"
git config --global user.email "your-email@example.com"
```

Replace the example email address with the appropriate email address before running the commands.

7.3. Clone the Application Repository

To download the Restaurant application repository, use its actual GitLab clone URL:

```
git clone <APPLICATION_REPOSITORY_URL>
cd restaurant-devops
```

Replace `<APPLICATION_REPOSITORY_URL>` with the repository's actual HTTPS or SSH clone URL.

If the repository has already been cloned, navigate to its existing local directory instead.

Verify the repository configuration:

```
git remote -v
git status
git branch --show-current
```

These commands display the configured remote repositories, the working tree status, and the current branch.

7.4. Git Development Workflow

The standard development workflow is:

1. Developers modify the application source code.
2. Changes are reviewed and tested.
3. Changes are committed to Git.
4. The changes are pushed to the remote repository.
5. Jenkins retrieves the relevant source code.
6. The CI/CD pipeline builds and tests the application.
7. Security scanning is performed.
8. A successful build produces a Docker image.
9. The image is published to the Docker Registry.
10. The deployment stage retrieves the approved image and deploys it to the application server.

The exact stages depend on the Jenkinsfile and pipeline configuration actually used in the project.

7.5. Common Git Commands

Check the current state of the repository:

```
git status
```

Review recent commits:

```
git log --oneline -10
```

Review changes that have not been committed:

```
git diff
```

Stage and commit changes:

```
git add .
git commit -m "Update Restaurant application"
```

Push the current branch:

```
git push
```

Before pushing, ensure that the intended files are staged and that no credentials, private keys, local environment files, or other secrets are included.

7.6. Docker Registry

Docker Registry is used to store and distribute container images. Instead of building the application independently on every deployment server, Jenkins can build an image and publish it to a registry. The deployment server can then retrieve the same image.

The previously used laboratory registry address was:

`192.168.20.159:5000`

This address must be verified before it is documented as the active registry endpoint.

The main benefits of using a registry are:

- Centralized storage of application images.
- Consistent application versions between build and deployment stages.
- Easier deployment and rollback to previously built images.

- Reduced need to rebuild the application on each target server.

7.7. Verify Docker Installation

On a server where Docker is installed, run:

```
docker --version
docker info
sudo systemctl status docker
```

These commands display the Docker version, engine information, and service status.

If Docker is not installed, the commands may fail. Installation instructions should be documented separately according to the operating system and the installation method used.

7.8. Build a Docker Image

A Docker image packages an application and its runtime dependencies into a reusable artifact.

From the application directory containing the Dockerfile, run:

```
docker build -t restaurant-app:latest .
```

This command builds an image named `restaurant-app` with the tag `latest`.

The command assumes that a valid Dockerfile exists in the current directory. The actual Dockerfile must match the Restaurant application's framework, dependencies, and startup command.

Verify that the image was created:

```
docker images
```

For reproducible deployments, versioned tags or immutable image digests are preferable to relying exclusively on the `latest` tag.

7.9. Tag the Image for the Registry

Tag the image with the registry address:

```
docker tag restaurant-app:latest 192.168.20.159:5000/restaurant-app:latest
```

This command assigns the image a name that includes the registry host and port.

The registry address is an example based on the previous laboratory configuration. Replace it if the active registry uses a different address.

7.10. Publish the Image

To upload the image to the registry:

```
docker push 192.168.20.159:5000/restaurant-app:latest
```

This operation requires the registry to be reachable and the client to have the necessary permissions.

If authentication is enabled, authenticate using the configured credentials before pushing. Credentials must not be written directly into scripts or committed to Git.

7.11. Verify Registry Connectivity

Check whether the registry endpoint responds:

```
curl -i --connect-timeout 5 http://192.168.20.159:5000/v2/
```

A Docker Registry API endpoint commonly returns HTTP `200 OK` for an accessible registry without authentication or HTTP `401 Unauthorized` when authentication is required. The exact response depends on the registry configuration.

A connection failure may indicate that the registry service is stopped, the address or port is incorrect, or network access is blocked.

Security note: The command above uses HTTP because the historical laboratory endpoint may have been configured without TLS. HTTP is not suitable for an exposed production registry. A production deployment should use HTTPS, authentication, and restricted network access. Docker clients may also require special configuration when connecting to an HTTP-only registry.

7.12. Pull the Image on the Deployment Server

On the target server, retrieve the image:

```
docker pull 192.168.20.159:5000/restaurant-app:latest
```

Verify the image:

```
docker images
```

This demonstrates the image distribution process between the build environment and the deployment environment.

The image name and tag must match the image published by the build stage.

7.13. Registry and Image Verification Checklist

- ☐ Git is installed and the repository can be accessed.
- ☐ The application repository URL has been verified.
- ☐ The current Git branch and working tree have been checked.
- ☐ The application contains a valid Dockerfile.
- ☐ Docker is installed and running on the required server.
- ☐ The application image builds successfully.
- ☐ The registry endpoint is reachable.
- ☐ The image can be pushed to the registry.
- ☐ The deployment server can pull the image.
- ☐ No credentials or secrets have been committed to the repository.

7.14. Expected Result

At the end of this stage, the source code is managed through Git, and the application can be packaged into a Docker image. The registry provides a central location from which the deployment environment can retrieve the application image.

These components establish the source control and artifact distribution foundation required for the Jenkins CI/CD pipeline described in the next section.

8. Jenkins CI/CD Pipeline

8.1. Introduction to Jenkins

Jenkins is an automation server used to implement Continuous Integration (CI) and Continuous Delivery or Deployment (CD).

In this project, Jenkins coordinates the process of obtaining application source code, building a Docker image, running validation and security checks, publishing the image to the Docker Registry, and initiating deployment.

The previously used Jenkins address was:

`http://192.168.20.162:8081`

The address and port must be verified against the active laboratory environment.

The Jenkins job previously associated with the project was named:

`restaurant-devops`

8.2. Continuous Integration and Continuous Deployment

Continuous Integration (CI) is the practice of frequently integrating code changes and automatically validating them through builds and tests.

Continuous Delivery (CD) prepares validated application versions for release. **Continuous Deployment** goes further by automatically deploying changes that satisfy the defined release conditions.

The distinction is important: a pipeline that builds and tests an application but requires manual approval before release implements a controlled delivery process, not fully automatic continuous deployment.

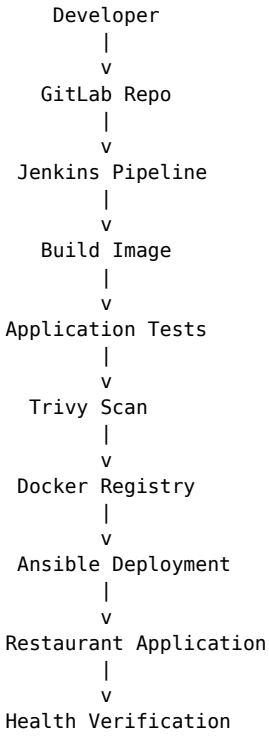
The intended workflow for this project is:

1. Retrieve source code from GitLab.
2. Check out the selected branch or commit.
3. Validate the application files.
4. Build the Docker image.
5. Run available application tests.
6. Scan the image for known vulnerabilities using Trivy.
7. Run dynamic application security testing using OWASP ZAP when the application is deployed to a test environment.
8. Publish the validated image to the Docker Registry.
9. Deploy the approved image to the target application server.
10. Verify that the application is responding correctly.
11. Record the result and retain enough information to support rollback.

Only the stages present in the actual Jenkins configuration should be reported as implemented. The list above describes the intended workflow.

8.3. Jenkins Architecture

Jenkins acts as the orchestration layer connecting source control, container tooling, security scanners, and deployment automation.



OWASP ZAP can be integrated as a separate dynamic security testing stage after the application has been deployed to an isolated test environment. It should not be assumed to run against production unless that behavior is explicitly configured and authorized.

8.4. Verify Jenkins Availability

Open the Jenkins web interface in a browser:

`http://192.168.20.162:8081`

On the Jenkins server, check whether the service is running:

```
sudo systemctl status jenkins
```

Check whether the expected port is listening:

```
sudo ss -lntp | grep ':8081'
```

Check the recent service logs:

```
sudo journalctl -u jenkins -n 100 --no-pager
```

These commands help identify whether Jenkins is running, listening on the expected port, and producing errors.

If Jenkins is running in a container rather than as a system service, use Docker commands to inspect the container instead.

8.5. Verify the Jenkins Job

In the Jenkins web interface:

1. Open the Jenkins dashboard.
2. Locate the `restaurant-devops` job.
3. Open the job.
4. Review its configuration and pipeline definition.
5. Open the most recent build.
6. Inspect the Console Output.
7. Record which stages succeeded and which failed.

The job name and configuration should be verified in Jenkins before being included as confirmed implementation details.

8.6. Jenkinsfile

A `Jenkinsfile` is a text file that defines the pipeline as code. Keeping the pipeline definition in the application repository makes it easier to review, version, and reproduce changes.

The following example is a **template**, not a claim about the contents of the existing pipeline. It assumes a working Docker installation, a valid Dockerfile, and an agent with the required tools and registry access.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'restaurant-app'
        REGISTRY = '192.168.20.159:5000'
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Validate Source') {
            steps {
                sh 'test -f Dockerfile'
                sh 'git status --short'
            }
        }

        stage('Build Image') {
            steps {
                sh '''
                    docker build \
                      -t ${REGISTRY}/${IMAGE_NAME}:${IMAGE_TAG} .
                '''
            }
        }

        stage('Trivy Scan') {
            steps {
                sh '''
                    trivy image \
                      --exit-code 1 \
                      --severity HIGH,CRITICAL \
                      ${REGISTRY}/${IMAGE_NAME}:${IMAGE_TAG}
                '''
            }
        }

        stage('Push Image') {
            steps {
```

```
                sh '''
                    docker push \
                      ${REGISTRY}/${IMAGE_NAME}:${IMAGE_TAG}
                '''
            }
        }

        stage('Deployment') {
            steps {
                echo 'Deployment must be configured for the target environment.'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully.'
        }

        failure {
            echo 'Pipeline failed. Review the Console Output.'
        }

        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

Important template notes

- The registry address must match the actual registry.
- The Jenkins agent must have Docker access and the required permissions.
- If the registry requires authentication, use Jenkins Credentials and an appropriate `docker login` step. Do not hardcode passwords or tokens.
- The Trivy stage requires Trivy to be installed on the agent or available through an approved execution method.
- The `Deployment` stage above is only a placeholder. It does not deploy the application.
- `checkout scm` requires the Jenkins job to be configured with the correct source repository.
- The source validation stage checks for the Dockerfile; it does not replace application-specific unit or integration tests.
- The sample uses a build-number tag to distinguish builds. Production deployments should also retain the source commit and image digest.
- The sample uses a simple registry configuration and does not establish that the current environment supports it.

Before using this template, compare it with the existing Jenkins job and adapt it to the actual application and server configuration.

8.7. Jenkins Credentials and Permissions

Jenkins may require credentials to access GitLab, Docker Registry, or remote deployment servers.

Credentials should be stored in the Jenkins Credentials system rather than directly in the `Jenkinsfile`.

Recommended practices:

1. Use a dedicated service account for automation.
2. Grant only the permissions required by the pipeline.

3. Use SSH keys or other supported secure authentication methods for remote deployment.
4. Avoid printing secrets in the build log.
5. Rotate credentials when necessary.
6. Never commit credentials to the source repository.

The Jenkins agent should have only the privileges necessary to build, scan, publish, and deploy the application.

8.8. Execute the Pipeline

To execute a configured pipeline:

1. Open the Jenkins dashboard.
2. Select `restaurant-devops`.
3. Click **Build Now**, if the job is configured for manual execution.
4. Open the new build.
5. Select **Console Output**.
6. Review each stage and identify the first failing command if the build fails.

If a webhook is configured, Jenkins may start automatically after a source code change is pushed. Do not assume webhook-based execution is enabled until it has been tested.

8.9. Verify the Pipeline Result

A successful pipeline should provide evidence that its configured stages completed successfully.

Examples of useful evidence include:

- Jenkins build number and final result.
- Git commit or branch used by the build.
- Docker image tag or digest.
- Application test output.
- Trivy scan results.
- Registry push output.
- Deployment logs.
- Application health-check results.

A green Jenkins build alone does not prove that every security, deployment, backup, or recovery requirement has been satisfied. Verification must correspond to the actual stages executed.

8.10. Troubleshooting

Problem: Jenkins is unavailable

Check:

```
sudo systemctl status jenkins
sudo journalctl -u jenkins -n 100 --no-pager
sudo ss -lntp | grep ':8081'
```

Verify the service status, configured port, firewall, and IP address.

Problem: Docker permission denied

Check:

```
docker info
id
```

The Jenkins execution user must have the required Docker permissions. Membership in the Docker group effectively grants highly privileged access to the host, so it should be treated as a security-sensitive permission.

Problem: Registry connection fails

Check:

```
curl -i --connect-timeout 5 http://192.168.20.159:5000/v2/
```

Verify the registry address, service status, network connectivity, authentication, and TLS configuration.

Problem: Trivy reports vulnerabilities

Review the affected packages, vulnerability identifiers, severity levels, and available fixes. Update vulnerable base images or application dependencies when appropriate, then rebuild and rescan the image.

A scan policy that fails builds on HIGH or CRITICAL findings should be documented together with any approved exceptions.

Problem: Pipeline fails because of insufficient disk space

Check:

```
df -h
docker system df
```

Remove only resources that are confirmed to be unnecessary. Avoid running broad Docker cleanup commands on a server that hosts important containers or images without first checking what would be deleted.

Problem: Deployment stage does not execute

Inspect the `Jenkinsfile`, job configuration, agent labels, credentials, and deployment logs. A placeholder stage or an `echo` command does not perform an actual deployment.

8.11. Verification Checklist

- ☐ Jenkins is accessible at its verified address.
- ☐ The `restaurant-devops` job exists and its configuration has been reviewed.
- ☐ Jenkins can retrieve the correct application repository.
- ☐ The application image builds successfully.
- ☐ Required application tests run successfully.
- ☐ Trivy scans the intended image.
- ☐ The image is published to the configured registry.
- ☐ The deployment stage performs the intended operation.
- ☐ The application is verified after deployment.
- ☐ Credentials are stored securely.
- ☐ Build failures are visible in the Jenkins Console Output.

8.12. Expected Result

Jenkins provides a repeatable automation process for the stages configured in the pipeline. It reduces manual work, improves consistency between builds, and provides a central record of build and deployment results.

The final implementation must be documented using the actual Jenkins job configuration and execution logs. Any stage that has not yet been configured or tested must be identified as pending rather than reported as completed.

9. Docker Deployment and Ansible Automation

9.1. Introduction to Ansible

Ansible is an automation and configuration management tool used to configure servers and execute repeatable deployment tasks.

Ansible typically connects to Linux servers through SSH and executes tasks defined in YAML playbooks. It does not require a separate Ansible agent on each managed Linux server.

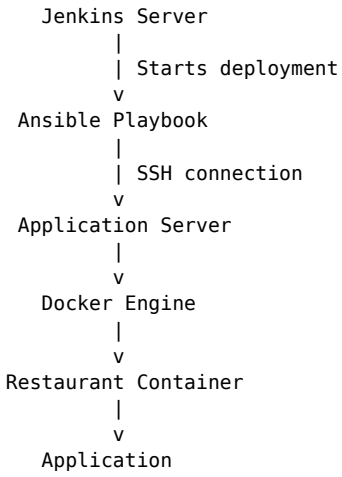
In this project, Ansible is intended to automate the deployment of the Restaurant application and reduce the need for manually executing commands on the target server.

The main responsibilities of Ansible include:

- Connecting to the application server.
- Verifying the target environment.
- Pulling the required Docker image.
- Creating or updating the application container.
- Configuring the application port mapping.
- Checking the deployed application's status.
- Supporting repeatable deployment and recovery procedures.

The exact tasks must match the playbooks used in the actual laboratory environment.

9.2. Ansible Architecture



Jenkins initiates the deployment process, Ansible executes the deployment tasks, and Docker runs the application container on the target server.

9.3. Verify Ansible Installation

On the machine where Ansible is installed, run:

```
ansible --version
```

Verify Python availability:

```
python3 --version
```

If Ansible is not installed, it must be installed using a method compatible with the host operating system. The installation procedure should be recorded separately according to the actual environment.

9.4. Inventory Configuration

An Ansible inventory defines the servers that Ansible manages.

A file named `inventory.ini` can contain the following example:

```
[app_servers]
restaurant-app ansible_host=192.168.20.39 ansible_user=hasmik
```

This is a template. Replace the IP address and SSH username with the actual target server details.

If the target server requires a different SSH user, configure the appropriate account. Do not assume that the local Linux username is also the correct remote username.

For a larger environment, multiple application servers can be listed:

```
[app_servers]
app1 ansible_host=192.168.20.41 ansible_user=deploy
app2 ansible_host=192.168.20.42 ansible_user=deploy
```

The addresses in this second example are illustrative and must not be treated as existing servers.

9.5. Test SSH Connectivity

Ansible uses SSH for typical Linux deployments. Before executing a playbook, verify that the automation machine can connect to the target server.

```
ssh hasmik@192.168.20.39
```

Replace the username and address with the values configured in the inventory.

For passwordless automation, an SSH key can be configured for the dedicated deployment account. The private key must remain protected and must never be committed to Git.

After confirming SSH access, test Ansible connectivity:

```
ansible -i inventory.ini app_servers -m ping
```

The Ansible `ping` module verifies that Ansible can connect to the managed hosts and execute its module. It is not an ICMP network ping.

A successful result normally includes:

```
"ping": "pong"
```

If the command fails, check the inventory, SSH credentials, Python availability, network access, and host permissions.

9.6. Docker Deployment Playbook

The following `deploy.yml` is a sample deployment template. It assumes that Docker is already installed on the target server, the registry is reachable, and the target user has the required permissions.

```
---
- name: Deploy Restaurant Application
  hosts: app_servers
```

```
  become: true
  gather_facts: true

  vars:
    registry: "192.168.20.159:5000"
    image_name: "restaurant-app"
    image_tag: "latest"
    container_name: "restaurant-app"
    host_port: 3001
    container_port: 3000

  tasks:
    - name: Display target operating system
      ansible.builtin.debug:
        msg: "Deploying on {{ ansible_facts['distribution'] }}"

    - name: Ensure Docker service is running
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true

    - name: Pull application image
      community.docker.docker_image:
        name: "{{ registry }}/{{ image_name }}"
        tag: "{{ image_tag }}"
        source: pull

    - name: Run Restaurant application container
      community.docker.docker_container:
        name: "{{ container_name }}"
        image: "{{ registry }}/{{ image_name }}:{{ image_tag }}"
        state: started
        restart_policy: unless-stopped
        published_ports:
          - "{{ host_port }}:{{ container_port }}"
```

Important requirements

This example requires the `community.docker` Ansible collection and a compatible Docker SDK for Python on the host where the Docker modules execute.

The collection can be installed with:

```
ansible-galaxy collection install community.docker
```

The exact runtime dependencies should be checked for the installed Ansible and Docker versions.

The target server must also have Docker installed and running. The playbook above starts the Docker service but does not install Docker itself.

If the registry requires authentication, configure credentials securely before attempting to pull the image.

The example uses the `latest` image tag for simplicity. A production deployment should use a versioned tag or immutable image digest so the exact deployed release can be identified and restored.

9.7. Execute the Deployment

From the directory containing `inventory.ini` and `deploy.yml`, run:

```
ansible-playbook -i inventory.ini deploy.yml
```

Ansible displays the result of each task and provides a summary at the end.

The summary includes:

- `ok` - tasks completed without requiring a change.
- `changed` - tasks modified the target system.
- `unreachable` - Ansible could not connect to a host.
- `failed` - one or more tasks failed.

A successful playbook execution does not by itself prove that the application is healthy. The deployed container and application endpoint must also be tested.

9.8. Verify the Docker Container

On the application server, run:

```
docker ps
```

This displays running containers.

To inspect the container configuration:

```
docker inspect restaurant-app
```

To view application logs:

```
docker logs --tail 100 restaurant-app
```

To check whether the application responds on the configured host port:

```
curl -i --connect-timeout 5 http://127.0.0.1:3001/
```

The application may use a different health endpoint. Replace `/` with the endpoint implemented by the application if necessary.

A response from `curl` is useful evidence, but the expected HTTP status and response body should be checked against the application's actual behavior.

9.9. Deployment Idempotency

Idempotency means that repeating an automation operation should not produce unnecessary or unintended changes when the system is already in the desired state.

Ansible modules such as `service` and `community.docker.docker_container` are designed to manage a desired state. For example, the playbook can ensure that Docker is running and that the application container exists with the configured image and port mapping.

However, the overall deployment is only as predictable as its configuration. Mutable image tags, external database changes, and application startup behavior can affect reproducibility.

Use versioned image tags and validate the deployed application after each release.

9.10. Rollback Strategy

Rollback restores a previously working application version after a failed deployment.

A basic rollback strategy is to retain the previous known-good image tag and redeploy that exact version.

For example, if a previously tested image was published as `restaurant-app:12`, the rollback deployment should reference that version rather than relying on `latest`.

The image must exist in the registry and be compatible with the application environment. Database migrations may require a separate recovery strategy because rolling back a container does not automatically reverse database schema changes or restore deleted data.

The exact rollback procedure should be tested before being described as implemented.

9.11. Troubleshooting

Ansible cannot connect to the target host

Check the inventory and test SSH access:

```
ansible -i inventory.ini app_servers -m ping -vvv
```

Review the error message for authentication failures, incorrect IP addresses, SSH host-key issues, or connectivity problems.

Docker image pull fails

Check registry connectivity and confirm that the image name and tag exist. Verify authentication and TLS settings if the registry requires them.

Docker container fails to start

Check:

```
docker ps -a
docker logs --tail 100 restaurant-app
```

Review the application logs, environment variables, port mapping, image version, and any missing runtime dependencies.

Application is not reachable

Check:

```
sudo ss -lntp
docker port restaurant-app
curl -i --connect-timeout 5 http://127.0.0.1:3001/
```

Confirm that the container listens on the expected internal port and that the host port is published correctly. Check firewall rules if remote clients cannot access the application.

9.12. Verification Checklist

- ☐ Ansible is installed on the automation host.
- ☐ The inventory contains the correct target server.
- ☐ SSH connectivity works.
- ☐ The Ansible `ping` module succeeds.
- ☐ The required Docker collection and dependencies are installed.
- ☐ The target server has Docker installed and running.
- ☐ The application image can be pulled from the registry.
- ☐ The deployment playbook completes successfully.
- ☐ The expected container is running.
- ☐ Application logs have been reviewed.
- ☐ The application endpoint responds as expected.
- ☐ A previous known-good image is retained for rollback.

9.13. Expected Result

Ansible provides a repeatable method for deploying the Restaurant application to a target server. By combining Jenkins, Ansible, Docker, and the Docker Registry, the project can reduce manual deployment work and improve consistency between application releases.

The inventory, playbook, image references, port mappings, and verification results must be updated to match the actual implementation before the project is submitted.

10. Security Scanning with Trivy and OWASP ZAP

10.1. Introduction

Security scanning helps identify potential vulnerabilities before an application is deployed to production.

This project uses two security testing tools:

- **Trivy** - scans container images and project files for known vulnerabilities and other security issues.
- **OWASP ZAP** - tests a running web application to identify potential security weaknesses.

These tools perform different types of testing and complement each other in the DevOps pipeline.

Important: The commands and configurations in this section are examples for the laboratory environment. Verify the actual installation, IP addresses, image names, and application URL before executing them.

10.2. Trivy - Container Image Vulnerability Scanning

10.2.1. Purpose

Trivy is a security scanner that can identify known vulnerabilities in operating system packages and supported application dependencies. Depending on the scan configuration, it can also detect exposed secrets and configuration problems.

In the project, Trivy can be integrated into Jenkins to scan a Docker image before it is published to the container registry.

The workflow is:

```
Git Repository -> Docker Build -> Trivy Scan -> Registry Push -> Deployment
```

If the scan identifies vulnerabilities that exceed the configured severity threshold, Jenkins can stop the pipeline.

10.2.2. Verify Trivy Installation

Run the following command on the machine where Trivy is expected to be installed:

```
trivy --version
```

If Trivy is installed, the command displays its version.

If the command is not found, Trivy must be installed before the scanning stage can run.

10.2.3. Scan a Docker Image

First, confirm that the image exists locally:

```
docker images
```

For example, if the application image is named `restaurant-app`, run:

```
trivy image --severity HIGH,CRITICAL restaurant-app:latest
```

If the image is stored in the private registry, use the complete image name:

```
trivy image --severity HIGH,CRITICAL 192.168.20.159:5000/restaurant-app:latest
```

The registry address above is a historical laboratory example. Replace it if the current registry uses a different address.

The scan reports vulnerabilities grouped by severity, such as:

- CRITICAL
- HIGH
- MEDIUM
- LOW
- UNKNOWN

The results depend on the image contents and the vulnerability database available to Trivy.

10.2.4. Generate a Security Report

Create a directory for reports:

```
mkdir -p security-reports
```

Generate a JSON report:

```
trivy image --format json --output security-reports/trivy-report.json restaurant-app:latest
```

Verify that the report was created:

```
ls -lh security-reports/
```

The JSON report can be retained as a project artifact or archived by Jenkins. It provides a record of the scan results for later review.

10.2.5. Configure a Security Gate

To make a Jenkins build fail when HIGH or CRITICAL vulnerabilities are found, use:

```
trivy image --exit-code 1 --severity HIGH,CRITICAL restaurant-app:latest
```

The `--exit-code 1` option makes Trivy return a non-zero exit code when matching vulnerabilities are found. Jenkins can use this result to mark the build as failed.

A failed scan does not automatically mean the application is exploitable. The findings must be reviewed, and the affected packages should be updated or otherwise addressed. Exceptions should be documented and approved rather than silently ignored.

10.3. OWASP ZAP - Dynamic Application Security Testing

10.3.1. Purpose

OWASP ZAP, also known as Zed Attack Proxy, is a web application security testing tool.

Unlike Trivy, which can scan a container image without running the application, ZAP tests an application while it is running.

A ZAP scan can identify potential issues such as:

- Missing or insufficient security headers.
- Cookies without recommended security attributes.
- Potentially unsafe responses or information disclosure.
- Other web security findings detected by its scanners.

The findings depend on the application, scan type, configuration, and pages that ZAP can reach. A clean scan does not prove that an application is completely secure.

10.3.2. Verify Application Availability

Before scanning, make sure the Restaurant Web Application is running.

For the historical laboratory configuration, the application was exposed on port `3001`, with container port `3000`.

Test the application from a machine that can access the application server:

```
curl -I http://192.168.20.39:3001
```

Replace the address with the actual application server IP and port if the environment has changed.

If the application is unavailable, resolve the connectivity or deployment problem before starting ZAP.

10.3.3. Run a ZAP Baseline Scan

The baseline scan is a suitable starting point for a laboratory project. It performs a short spider and passive security checks. It is not equivalent to a comprehensive penetration test or a full active scan.

Create a directory for ZAP reports:

```
mkdir -p zap-reports
```

Run the baseline scan using the official ZAP container image:

```
docker run --rm -t \
  -v "$PWD/zap-reports:/zap/wrk:rw" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t http://192.168.20.39:3001 \
  -r zap-report.html
```

This example assumes that Docker can pull the ZAP image and that the ZAP container can reach the application at the specified address.

The report is written to `zap-reports/zap-report.html` on the machine where the command is executed.

Verify the output:

```
ls -lh zap-reports/
```

Open the HTML report in a browser to review the findings.

Safety note: Run security scans only against systems you own or have explicit permission to test. Active scans can send potentially disruptive requests and may change application data. Use an isolated test environment and test data for more aggressive scans.

10.3.4. Interpret ZAP Results

ZAP findings generally include a risk rating, description, affected URL, and recommended remediation.

For each finding:

1. Review the affected URL and the evidence.
2. Determine whether the finding applies to the application.

3. Reproduce and investigate it in the test environment when appropriate.
4. Fix confirmed issues.
5. Run the scan again to verify the result.
6. Document any accepted exceptions and their justification.

Automated scanners can produce false positives. Findings should be validated rather than treated as confirmed vulnerabilities automatically.

10.4. Integrating Security Scanning into Jenkins

Trivy can be added to the Jenkins pipeline after the Docker image is built and before it is pushed to the registry.

Example stage:

```
stage('Trivy Security Scan') {
    steps {
        sh '''
            trivy image \
              --exit-code 1 \
              --severity HIGH,CRITICAL \
              restaurant-app:${BUILD_NUMBER}
        '''
    }
}
```

This example assumes that the image was built using the Jenkins build number as its tag. The image name and tag must match the actual build command.

If the scan fails, Jenkins stops the pipeline at that stage unless the pipeline is explicitly configured to handle the failure differently.

OWASP ZAP can be added as a separate stage after deploying the application to a test environment. The ZAP target URL must be reachable from the machine or container running the scan.

For reliable reporting, configure Jenkins to archive the generated Trivy and ZAP reports. Report archiving and the ZAP pipeline stage must be configured and tested before they can be considered implemented.

10.5. Troubleshooting

Trivy cannot find the image

Check available images:

```
docker images
```

Confirm that the image name and tag match the build output. If necessary, build the image or pull the correct tag from the registry.

ZAP cannot reach the application

Test the application URL from the machine running the scan:

```
curl -I http://192.168.20.39:3001
```

Check the application container and listening ports:

```
docker ps
sudo ss -lntp
```

Confirm that firewall rules permit the required connection and that the URL uses the correct host IP and port.

ZAP reports "No space left on device"

Check available disk space:

```
df -h
docker system df
du -sh zap-reports
```

Review Docker storage and existing reports before removing files. Do not delete containers, images, or volumes indiscriminately, because they may be needed by the project.

The report was not created

Check the command output for errors and verify that the report directory exists and is writable. Confirm that the container exited and that the output path is mounted correctly.

10.6. Security Testing Checklist

- ☐ Verify that Trivy is installed and working.
- ☐ Scan the Restaurant application image.
- ☐ Review HIGH and CRITICAL findings.
- ☐ Generate and retain the Trivy report.
- ☐ Verify that the Restaurant application is reachable.
- ☐ Run an authorized ZAP baseline scan against the test application.
- ☐ Review the ZAP HTML report.
- ☐ Investigate and remediate confirmed findings.
- ☐ Repeat scans after fixes.
- ☐ Verify that Jenkins fails the build when the configured Trivy security gate is triggered.
- ☐ Archive scan reports in Jenkins if report archiving has been configured.

10.7. Expected Result

The security-testing stage should provide documented evidence that the Restaurant application image can be scanned for known vulnerabilities and that the running web application can be checked for potential security weaknesses.

The intended pipeline should prevent an image from progressing when it violates the configured security gate. OWASP ZAP results should be reviewed and addressed before production deployment.

Actual completion must be demonstrated by the scan output, generated reports, and Jenkins build results. Installing the tools or adding example commands to the documentation alone does not establish that the scans have been successfully executed.

11. High Availability and Load Balancing with HAProxy and Keepalived

11.1. Introduction

High availability (HA) is a system design approach intended to reduce downtime when a server or service fails.

In this project, HAProxy and Keepalived are included in the target architecture to improve the availability of the Restaurant Web Application.

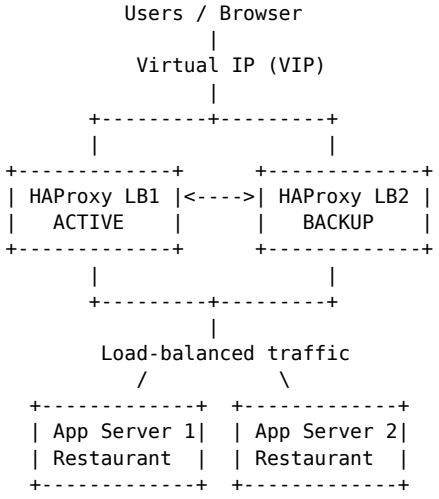
- **HAProxy** distributes incoming HTTP traffic between multiple application servers.
- **Keepalived** can provide a Virtual IP (VIP) that moves between two load balancer servers when the active load balancer becomes unavailable.

These tools solve different problems. HAProxy handles traffic distribution, while Keepalived supports failover between load balancers.

Implementation status: The configuration examples below describe the target design. They are not proof that high availability has already been deployed. Actual server addresses, network interfaces, and test results must be verified in the laboratory.

11.2. Target Architecture

The planned architecture contains two load balancers and two application servers.



Keepalived manages the VIP between the load balancers. HAProxy on the active load balancer forwards requests to healthy application servers.

This design requires two separate load balancer hosts and two application hosts. Running all components on one machine does not provide the same protection against host failure.

11.3. Prerequisites

Before configuring high availability, verify the following:

1. Two load balancer machines are available.
2. Two application servers are available and can run the Restaurant application.
3. Both application servers expose the application on the same port, such as `3001`.

4. The load balancers can reach both application servers.
5. The two load balancers share a network that supports the chosen VIP failover method.
6. The necessary firewall rules permit application traffic and Keepalived health/failover communication.

The IP addresses below are illustrative and must be replaced with addresses from the actual network.

Component	Example address
Load Balancer 1	`192.168.20.41`
Load Balancer 2	`192.168.20.42`
Application Server 1	`192.168.20.43`
Application Server 2	`192.168.20.44`
Virtual IP	`192.168.20.50`

The VIP must be unused and belong to the appropriate subnet. Do not assign these example addresses without checking for conflicts.

11.4. Install HAProxy

Run the following commands on both load balancer machines:

```
sudo dnf install -y haproxy
sudo systemctl enable haproxy
```

Verify the installation:

```
haproxy -v
```

Back up the existing configuration before making changes:

```
sudo cp -a /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
```

11.5. Configure HAProxy

Edit `/etc/haproxy/haproxy.cfg` using an editor available on the system. The following is an example configuration for HTTP traffic.

```
global
    log /dev/log local0
    log /dev/log local1 notice
    maxconn 2000

defaults
    log global
    mode http
    option httplog
    option httpchk GET /
    timeout connect 5s
    timeout client  30s
    timeout server  30s

frontend restaurant_frontend
    bind *:80
    default_backend restaurant_servers

backend restaurant_servers
    balance roundrobin
    server restaurant1 192.168.20.43:3001 check
    server restaurant2 192.168.20.44:3001 check
```

Configuration explanation

- `frontend restaurant_frontend` defines the entry point for incoming HTTP requests.
- `bind *:80` listens on port 80 on the load balancer.
- `default_backend restaurant_servers` sends requests to the configured backend pool.
- `balance roundrobin` distributes requests in rotation.
- `server ... check` defines an application server and enables health checking.

The health check above requests `/`. If the application uses a different health endpoint, update the check accordingly. A simple HTTP response does not always prove that all application dependencies are healthy.

Validate the configuration before restarting HAProxy:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

If validation succeeds, enable and start the service:

```
sudo systemctl enable --now haproxy
sudo systemctl status haproxy --no-pager
```

Test the load balancer locally:

```
curl -I http://127.0.0.1/
```

The test will succeed only if HAProxy is listening and at least one reachable backend can respond appropriately.

Production note: This example uses plain HTTP. Production deployments should configure HTTPS, certificates, secure management access, and appropriate security policies.

11.6. Install Keepalived

Keepalived must be installed on both load balancer machines:

```
sudo dnf install -y keepalived
```

Check the network interface name on each machine:

```
ip -br address
```

Identify the interface connected to the network containing the VIP. The interface name may differ between servers; do not assume it is `ens33`.

11.7. Configure Keepalived

The following configuration is a template. Replace the interface, addresses, and priority values with the actual network details.

Load Balancer 1 - Preferred Active Node

Example `/etc/keepalived/keepalived.conf`:

```
global_defs {
    router_id RESTAURANT_LB1
}

vrrp_script chk_haproxy {
    script "/usr/bin/killall -0 haproxy"
    interval 2
    weight -20
}

vrrp_instance VI_RESTAURANT {
```

```
    state MASTER
    interface ens33
    virtual_router_id 51
    priority 150
    advert_int 1

    authentication {
        auth_type PASS
        auth_pass ChangeThisPassword
    }

    virtual_ipaddress {
        192.168.20.50/24
    }

    track_script {
        chk_haproxy
    }
}
```

Load Balancer 2 - Backup Node

Use the same virtual router ID, authentication configuration, VIP, and network interface as appropriate for the shared network. Change the node-specific settings as shown:

```
global_defs {
    router_id RESTAURANT_LB2
}

vrrp_script chk_haproxy {
    script "/usr/bin/killall -0 haproxy"
    interval 2
    weight -20
}

vrrp_instance VI_RESTAURANT {
    state BACKUP
    interface ens33
    virtual_router_id 51
    priority 100
    advert_int 1

    authentication {
        auth_type PASS
        auth_pass ChangeThisPassword
    }

    virtual_ipaddress {
        192.168.20.50/24
    }

    track_script {
        chk_haproxy
    }
}
```

These configurations demonstrate the general arrangement; they must be validated in the actual environment. The example process check requires the `killall` command to be available. A production configuration should use a reliable health-check script that detects whether HAProxy is actually able to serve traffic, not merely whether its process exists.

VRRP advertisements and virtual IP movement must be supported by the network and permitted by the firewall. Some virtualized or cloud networks restrict this behavior and require a different failover method. The example authentication setting is not a substitute for network security.

11.8. Start and Verify Keepalived

After saving and validating the configuration on both load balancers, run:

```
sudo systemctl enable --now keepalived
sudo systemctl status keepalived --no-pager
```

Check the assigned addresses:

```
ip -br address
```

On the active node, the VIP should appear when Keepalived has successfully established the VRRP state. On the backup node, it should normally not appear while the active node is healthy.

Test access through the VIP:

```
curl -I http://192.168.20.50/
```

Replace the example VIP if necessary.

11.9. High-Availability Tests

Perform these tests only in the laboratory environment, preferably during a maintenance window.

Test 1 - Normal Operation

1. Confirm that both application servers are running.
2. Confirm that HAProxy is active on both load balancers.
3. Confirm that the VIP is assigned to one load balancer.
4. Access the application through the VIP.
5. Review HAProxy and application logs.

Expected result: requests through the VIP reach a healthy application server.

Test 2 - Application Server Failure

1. Stop the Restaurant application on one application server.
2. Send requests through the VIP.
3. Check whether HAProxy marks the failed backend as unavailable.
4. Restore the application server.

Expected result: requests can continue through the remaining healthy backend, provided that it is available and the application supports the request being tested.

Test 3 - Active Load Balancer Failure

1. Confirm that the VIP is assigned to Load Balancer 1.
2. Stop Keepalived on Load Balancer 1 in the test environment:

```
sudo systemctl stop keepalived
```

3. Check whether the VIP moves to Load Balancer 2.
4. Test the application through the VIP.
5. Restore the service on Load Balancer 1:

```
sudo systemctl start keepalived
```

Expected result: Load Balancer 2 takes over the VIP if VRRP and network failover are configured correctly.

A short interruption may occur during failover. Actual recovery time must be measured rather than assumed.

11.10. Troubleshooting

HAProxy fails to start

Check the configuration:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Review the service logs:

```
sudo journalctl -u haproxy -n 100 --no-pager
```

Backend servers are unavailable

Test each backend directly:

```
curl -I http://192.168.20.43:3001/
curl -I http://192.168.20.44:3001/
```

Confirm that the application is running, the port is correct, and the firewall permits the connection.

The VIP does not move

Check Keepalived status and logs:

```
sudo systemctl status keepalived --no-pager
sudo journalctl -u keepalived -n 100 --no-pager
ip -br address
```

Verify the interface name, VIP subnet, VRRP settings, network restrictions, and firewall rules on both nodes.

11.11. Limitations and Considerations

High availability at the load balancer and application layers does not automatically make the entire system highly available.

The following risks must also be considered:

- Both application servers may depend on the same database.
- A shared storage or network failure can affect both application servers.
- Application sessions and uploaded files may require shared or replicated storage.
- Database failover must be designed separately.
- Monitoring and backup systems must remain available and recoverable.
- HTTPS certificates and configuration changes must be managed consistently across both load balancers.

The complete solution should be tested for individual service failures as well as broader infrastructure failures.

11.12. Completion Checklist

- ☐ Two load balancer hosts are available.
- ☐ HAProxy is installed and its configuration validates on both hosts.
- ☐ Both application servers are reachable from the load balancers.
- ☐ Keepalived is configured on both load balancers.
- ☐ The VIP is assigned to the expected node.
- ☐ Requests through the VIP reach the Restaurant application.

- ☐ Application backend failure has been tested.
- ☐ Load balancer failover has been tested.
- ☐ Logs and observed recovery behavior have been documented.

11.13. Expected Result

The target architecture provides a single virtual entry point for users, distributes requests across multiple application servers, and supports VIP failover between two load balancers.

Successful completion must be demonstrated through configuration validation, service status, VIP assignment, application access, and recorded failure-test results. If the required servers or network capabilities are unavailable, document this section as the proposed high-availability design rather than claiming that it is operational.

12. PostgreSQL Replication, Backup, and Recovery

12.1. Introduction

The Restaurant Web Application requires a database to store and retrieve application data. PostgreSQL is used as the relational database management system in this project.

Database availability and data protection are essential because a web application may continue running while losing access to its data.

This section covers three important concepts:

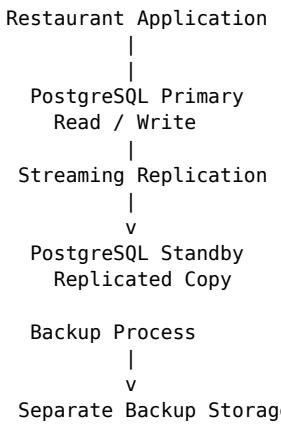
- **Database replication:** Maintains a copy of database changes on another PostgreSQL server.
- **Backup:** Creates a recoverable copy of database data.
- **Recovery:** Restores data after accidental deletion, corruption, or infrastructure failure.

Replication and backup serve different purposes. Replication can improve availability, but accidental changes or deletions may also reach the replica. Backups provide separate recovery points.

Implementation status: The configurations below are templates for the target architecture. The actual PostgreSQL version, database name, credentials, network addresses, and replication status must be verified in the laboratory before this section can be marked as implemented.

12.2. Target Architecture

The proposed design uses a primary PostgreSQL server and a standby replica.



The primary accepts database changes. The standby receives changes through PostgreSQL streaming replication.

A separate backup process stores database backups in a location that is independent of the primary server.

Streaming replication alone does not guarantee automatic failover. A complete automatic failover design requires additional orchestration, health checks, fencing or split-brain protection, and a controlled method for promoting the standby.

12.3. Verify PostgreSQL

On the database server, check the PostgreSQL client version:

```
psql --version
```

If PostgreSQL runs as a system service, check its status:

```
sudo systemctl status postgresql --no-pager
```

Service names can differ by operating system and PostgreSQL package. If PostgreSQL runs in Docker, use:

```
docker ps
```

Connect to PostgreSQL using an authorized database account:

```
sudo -u postgres psql
```

Inside the PostgreSQL prompt, inspect the server version and databases:

```
SELECT version();
\l
```

Exit with:

```
\q
```

Record the actual PostgreSQL version, database name, and deployment method in the project documentation.

12.4. Configure Streaming Replication

PostgreSQL streaming replication requires compatible primary and standby servers, network connectivity, replication permissions, and appropriate configuration.

The following examples use PostgreSQL configuration files typically named `postgresql.conf` and `pg_hba.conf`. Their paths vary by operating system and installation method.

12.4.1. Configure the Primary Server

On the primary PostgreSQL server, update the relevant settings in `postgresql.conf`:

```
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
```

These settings enable the server to generate the required write-ahead log information and support replication connections.

`listen_addresses = '*'` allows PostgreSQL to listen on all available network interfaces. Restrict network access with firewall rules and `pg_hba.conf`; for a production system, bind only to the required interfaces where practical.

Restart PostgreSQL if required by the configuration change:

```
sudo systemctl restart postgresql
```

Use the actual service name for the installed PostgreSQL version.

12.4.2. Create a Replication Account

Connect to the primary server as a PostgreSQL administrator:

```
sudo -u postgres psql
```

Create a dedicated replication account:

```
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'REPLACE_WITH_A_STRONG_SECRET';
```

Replace the example password with a strong secret. Do not store production credentials in a public repository or documentation file.

Exit the PostgreSQL prompt with `\q`.

12.4.3. Allow Replication Connections

On the primary server, add an appropriately restricted rule to `pg_hba.conf`.

Example only:

```
host    replication    replicator    192.168.20.45/32    scram-sha-256
```

Here, `192.168.20.45` represents the example standby address. Replace it with the real standby IP.

The `scram-sha-256` method requires compatible password configuration and clients. Use the authentication method supported by the installed PostgreSQL configuration and follow current security recommendations.

Reload PostgreSQL after modifying `pg_hba.conf`:

```
sudo systemctl reload postgresql
```

Verify firewall rules permit the standby to connect to the primary's PostgreSQL port, normally TCP `5432`. Do not expose the database port publicly.

12.4.4. Initialize the Standby

The standby must begin with a consistent copy of the primary database. A common approach is to use `pg_basebackup`.

On the standby server, stop PostgreSQL before preparing the data directory. The exact service name and data directory depend on the installed PostgreSQL version.

```
sudo systemctl stop postgresql
```

Warning: Never delete or overwrite an existing PostgreSQL data directory without verifying its contents and making a safe backup. The following is a procedure outline, not a universal copy-paste command.

Use `pg_basebackup` to initialize an empty standby data directory from the primary. For example, when the environment and PostgreSQL version support this syntax:

```
pg_basebackup \
  -h 192.168.20.37 \
  -p 5432 \
  -U replicator \
  -D /path/to/standby/data \
  -Fp \
  -Xs \
  -P \
  -R
```

Replace the primary address and data directory with the actual values. This command assumes the target data directory is suitable for initialization and that the replication account can authenticate.

The `-R` option writes the standby connection settings and creates the standby signal file for supported PostgreSQL versions. A secure authentication method must be configured, such as an appropriately permissioned password file or another supported credential mechanism. Avoid putting real passwords directly in shell commands.

Start PostgreSQL on the standby after initialization:

```
sudo systemctl start postgresql
```

Verify that the service starts and that the standby connects to the primary.

12.5. Verify Replication

On the Primary

Connect to PostgreSQL and run:

```
SELECT
    application_name,
    client_addr,
    state,
    sync_state
FROM pg_stat_replication;
```

A connected standby should appear in the results. The output depends on the configured replication mode and connection status.

On the Standby

Run:

```
SELECT pg_is_in_recovery();
```

A result of `true` indicates that the server is operating in recovery/standby mode.

Replication status should be monitored continuously. An empty result from `pg_stat_replication` may indicate that the standby is disconnected, incorrectly configured, or not yet initialized.

12.6. Create a Logical Database Backup

A logical backup stores database objects and data in a portable dump format.

For example, to back up a database named `restaurant_db`:

```
pg_dump -U postgres -h 127.0.0.1 -d restaurant_db -F c -f restaurant_db.dump
```

Replace the database name, host, and user as appropriate. This example assumes the PostgreSQL account can authenticate from the current machine.

For a local administrative connection, an alternative is:

```
sudo -u postgres pg_dump -d restaurant_db -F c -f restaurant_db.dump
```

Check that the backup file exists:

```
ls -lh restaurant_db.dump
```

A successful `pg_dump` command and a non-empty file are useful initial checks, but they do not by themselves prove that the backup can be restored.

Backup a Database Running in Docker

If PostgreSQL runs in a container, identify the correct container:

```
docker ps
```

A database dump can be written from the container to a file on the host. For example, when the container and database names are known:

```
docker exec -t POSTGRES_CONTAINER_NAME \
  pg_dump -U postgres -d restaurant_db -F c \
  > restaurant_db.dump
```

Replace `POSTGRES_CONTAINER_NAME` with the actual container name. Ensure the command succeeds and check the generated file. For sensitive or production databases, use secure credential handling and protect the backup file.

12.7. Restore a Database Backup

Restoration should first be tested in a separate test database or isolated environment.

Create a new empty database:

```
createdb -U postgres restaurant_restore_test
```

Restore the custom-format backup:

```
pg_restore -U postgres \
  -d restaurant_restore_test \
  --verbose \
  restaurant_db.dump
```

Verify the restored database by connecting to it and checking its tables and expected data.

For example:

```
psql -U postgres -d restaurant_restore_test
```

Inside the PostgreSQL prompt:

```
\dt
```

The actual tables depend on the Restaurant application schema. Run application-level checks as well as database checks before accepting the restoration as successful.

Do not restore over a live production database without an approved recovery procedure and a verified backup.

12.8. Backup Retention and Storage

Backups should be stored separately from the primary database server so that a host failure does not destroy both the original data and its backups.

A backup policy should define:

- Backup frequency.
- Retention period.
- Storage location.
- Access permissions and encryption.
- Monitoring and failure notifications.
- Restore-test frequency.
- Recovery Point Objective (RPO).
- Recovery Time Objective (RTO).

RPO describes how much data loss, measured in time, the organization can tolerate. **RTO** describes how long recovery may take before the service must be restored.

The required values should be based on the project's objectives, not chosen arbitrarily.

A scheduled backup can be automated with a systemd timer or cron job, but the schedule must also check the exit status, preserve logs, prevent unsafe overwrites, and alert when a backup fails. Scheduling a command alone is not a complete backup solution.

12.9. Recovery and Failover Considerations

If the primary PostgreSQL server fails, the standby may be promoted to become the new primary. Promotion must be a controlled operation.

Before promotion:

1. Determine whether the original primary is truly unavailable.
2. Prevent both servers from accepting independent writes.
3. Confirm the standby is sufficiently up to date.
4. Follow the documented promotion procedure.
5. Update application connection settings or the database endpoint.
6. Verify database writes and application functionality.
7. Record the recovery time and any data loss.

Do not promote a standby simply because a connection check failed. A network interruption can make a healthy primary appear unavailable, and promoting the standby without protecting against the original primary returning can cause split-brain operation.

Automatic database failover requires additional orchestration and testing; it is not provided by streaming replication alone.

12.10. Troubleshooting

The standby cannot connect to the primary

Check the network and PostgreSQL port:

```
nc -vz 192.168.20.37 5432
```

If `nc` is not installed, use an available network diagnostic tool. Verify `listen_addresses`, the replication rule in `pg_hba.conf`, firewall rules, and credentials.

Replication is not visible

On the primary, inspect:

```
SELECT * FROM pg_stat_replication;
```

On both servers, review PostgreSQL logs and confirm that the replication connection is active.

A backup fails

Check the command's exit code, available disk space, file permissions, database connectivity, and authentication configuration.

```
df -h
```

A restore fails

Check that the backup is readable, the PostgreSQL versions are compatible, the target database exists where required, and the restoring account has sufficient permissions. Test with a fresh, isolated database before attempting production recovery.

12.11. Completion Checklist

- ☐ PostgreSQL version and database name documented.
- ☐ Primary server configuration verified.

- ☐ Replication account and access rules configured securely.
- ☐ Standby initialized from a consistent primary copy.
- ☐ Replication state verified on both servers.
- ☐ Logical database backup created.
- ☐ Backup stored separately from the primary server.
- ☐ Backup restoration tested in an isolated database.
- ☐ Backup retention and monitoring requirements documented.
- ☐ Failover limitations and recovery steps documented.

12.12. Expected Result

The target design provides a PostgreSQL primary with a standby replica and a separate database backup process. These mechanisms support database availability and recovery from different failure scenarios.

Successful completion must be demonstrated by replication status, a generated backup, and a successful restoration test. Do not claim automatic failover, a specific recovery time, or guaranteed data-loss protection unless those capabilities have been implemented and tested.

13. Monitoring and Alerting with Prometheus and Grafana

13.1. Introduction

Monitoring is an important part of a DevOps infrastructure because it helps administrators identify performance problems, service failures, and resource exhaustion before they cause prolonged downtime.

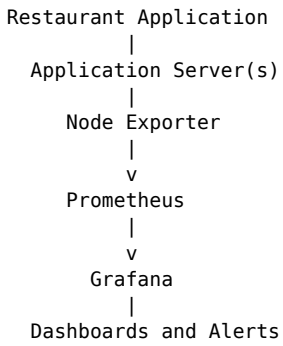
This project uses Prometheus and Grafana as the target monitoring solution for the Restaurant Web Application infrastructure.

- **Prometheus** collects and stores time-series metrics from configured targets.
- **Grafana** displays metrics in dashboards and supports alerting when configured conditions are met.
- **Node Exporter** exposes operating system metrics such as CPU, memory, disk, and network activity.
- **Alert rules** define conditions that indicate a possible problem.

Monitoring complements logging and security scanning. Metrics help identify what is happening, while logs provide additional details that help explain why.

Implementation status: The configurations below are examples for the target architecture. Verify installed versions, server addresses, ports, and service status before claiming that monitoring is operational.

13.2. Target Architecture



Prometheus periodically scrapes metrics from configured targets. Grafana connects to Prometheus as a data source and visualizes the collected information.

Node Exporter provides host-level metrics. Application-specific metrics require an appropriate exporter or instrumentation in the application itself.

13.3. Monitoring Requirements

The monitoring system should cover the following areas:

Component	Metrics or checks
Linux servers	CPU, memory, disk, and network
Restaurant application	Availability and response behavior
HAProxy	Backend health and traffic, if the required metrics endpoint is configured

Component	Metrics or checks
PostgreSQL	Database availability and replication health, using suitable checks or an exporter
Jenkins	Build results and job status, through appropriate integration
Backup process	Job success, failure, and last successful backup
Prometheus	Target health and monitoring service status

Not all these metrics are collected automatically. Each target needs the relevant exporter, integration, probe, or custom check.

13.4. Install Prometheus and Node Exporter

Prometheus can be installed as a native service or run in a container. For a repeatable laboratory setup, the following example uses Docker Compose.

First, verify Docker:

```
docker --version
docker compose version
```

Create a directory for the monitoring configuration:

```
mkdir -p ~/restaurant-monitoring
cd ~/restaurant-monitoring
```

Create the following file named `prometheus.yml`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]

  - job_name: node_exporter
    static_configs:
      - targets: ["node-exporter:9100"]
```

This configuration collects Prometheus's own metrics and metrics from Node Exporter running in the same Compose network.

To monitor other servers, add their reachable Node Exporter addresses under the relevant target configuration. The exporter must be installed and running on each monitored server.

13.5. Configure Docker Compose

Create a file named `compose.yml` in the same directory:

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: restaurant-prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"
    restart: unless-stopped

  node-exporter:
```

```
    image: prom/node-exporter:latest
    container_name: restaurant-node-exporter
    ports:
      - "9100:9100"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: restaurant-grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    restart: unless-stopped

volumes:
  prometheus_data:
  grafana_data:
```

This is a basic laboratory example. The Node Exporter container shown here exposes its own container metrics and should not be treated as a complete view of the host operating system. To monitor the actual Linux host, Node Exporter must be configured with appropriate host filesystem and namespace access, or installed directly as a host service. Those settings vary by operating system and security requirements.

For production, pin image versions rather than relying on `latest`, restrict access to monitoring ports, and configure authentication and secure access.

Start the services:

```
docker compose up -d
```

Check their status:

```
docker compose ps
```

Review logs if a service fails:

```
docker compose logs --tail=100
```

13.6. Verify Prometheus

Open the Prometheus interface in a browser:

`http://MONITORING_SERVER_IP:9090`

Replace `MONITORING_SERVER_IP` with the actual monitoring server address.

Open **Status > Targets** and check the configured scrape targets.

Targets should show an `UP` state when Prometheus can successfully collect their metrics. A target in the `DOWN` state requires investigation.

You can also test a basic Prometheus metric from the browser's query interface:

```
up
```

The `up` metric reports whether a configured target was successfully scraped. A value of `1` means the most recent scrape succeeded; `0` means it failed.

13.7. Configure Grafana

Open Grafana:

`http://MONITORING_SERVER_IP:3000`

Sign in using the credentials configured for the installation. For a new laboratory instance, follow the initial login instructions and set a strong administrator password.

Add Prometheus as a Data Source

1. Open **Connections > Data sources**.
2. Select **Add data source**.
3. Choose **Prometheus**.
4. Enter the Prometheus URL.
5. Save and test the configuration.

If Grafana and Prometheus are running in the same Compose network defined above, use:

`http://prometheus:9090`

The service name works because Docker Compose provides internal DNS for services on the same network.

If Grafana is installed on a different machine, use a reachable Prometheus URL and ensure access is securely restricted.

Create a Dashboard

Create a dashboard and add panels using Prometheus queries.

Examples:

Target availability

```
up
```

Prometheus process memory

```
process_resident_memory_bytes
```

Scrape duration

```
scrape_duration_seconds
```

These queries demonstrate basic Prometheus metrics. They do not provide complete host CPU, memory, or disk monitoring unless the corresponding host metrics are being collected.

For host monitoring, configure a suitable Node Exporter installation and use metrics such as `node_cpu_seconds_total`, `node_memory_MemAvailable_bytes`, and `node_filesystem_avail_bytes`.

13.8. Configure a Basic Alert Rule

Prometheus can evaluate alert rules and expose alerts through its interface. Grafana also supports alert rules and notification integrations.

For example, create a file named `alert-rules.yml`:

```
groups:
  - name: restaurant-monitoring
    rules:
      - alert: MonitoringTargetDown
        expr: up == 0
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "A monitoring target is unavailable"
          description: "Prometheus has been unable to scrape a configured target for at least two
              minutes."
```

Update `prometheus.yml` to load the rule file:

```
rule_files:
  - "alert-rules.yml"
```

Mount `alert-rules.yml` into the Prometheus container in `compose.yml`:

```
volumes:
  - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
  - ./alert-rules.yml:/etc/prometheus/alert-rules.yml:ro
  - prometheus_data:/prometheus
```

Then recreate or restart Prometheus so the new configuration is loaded:

```
docker compose up -d
```

Validate the configuration and check Prometheus logs if the service does not start correctly.

The example rule detects failed scrapes, but it does not send email or other notifications by itself. For notifications, configure Grafana alert contact points or a Prometheus Alertmanager integration and test delivery.

13.9. Monitoring Best Practices

A monitoring solution should provide useful signals without overwhelming administrators with unnecessary alerts.

Recommended practices include:

- Monitor service availability and infrastructure resource usage.
- Set thresholds based on observed baseline behavior.
- Track disk space to detect possible failures caused by storage exhaustion.
- Monitor database replication lag and backup job results through appropriate integrations.
- Alert on sustained failures rather than every short-lived fluctuation.
- Protect Grafana, Prometheus, and exporter endpoints from unauthorized access.
- Retain monitoring data for an appropriate period.
- Document alert ownership and response procedures.
- Test notification delivery and alert recovery.

A dashboard alone does not guarantee operational monitoring. Alerts must be configured, delivered, reviewed, and connected to a response process.

13.10. Troubleshooting

A container is not running

Check the service status and logs:

```
docker compose ps
docker compose logs --tail=100 prometheus
docker compose logs --tail=100 grafana
docker compose logs --tail=100 node-exporter
```

Prometheus target is down

Verify the target address and port, check network connectivity, and confirm that the exporter is running. Remember that container service names are available only to containers sharing the relevant network.

Grafana cannot connect to Prometheus

Check the data source URL. If both services use the same Compose network, the URL should generally use the Prometheus service name rather than `localhost`.

A monitoring port is inaccessible

Check listening ports and firewall rules:

```
sudo ss -lntp
```

Confirm that the service is bound to the expected interface and that the required access is permitted. Do not expose administrative monitoring interfaces directly to the public internet without suitable protection.

Monitoring storage fills up

Check available disk space:

```
df -h
docker system df
```

Review retention settings and data growth. Do not delete monitoring volumes indiscriminately because they contain historical metrics.

13.11. Completion Checklist

- ☐ Prometheus is installed and running.
- ☐ Prometheus configuration is valid.
- ☐ At least one monitoring target is successfully scraped.
- ☐ Grafana is installed and running.
- ☐ Prometheus is configured as a Grafana data source.
- ☐ A dashboard displays collected metrics.
- ☐ At least one alert rule has been configured.
- ☐ Alert notifications are configured and tested if required.
- ☐ Monitoring access is appropriately restricted.
- ☐ Logs and troubleshooting procedures are documented.

13.12. Expected Result

The target monitoring solution collects metrics through Prometheus and displays them in Grafana dashboards. Configured alert rules help identify unavailable monitoring targets and other operational problems when the necessary metrics are available.

Successful completion must be demonstrated by running services, healthy Prometheus targets, working Grafana dashboards, and tested alert behavior. The example configuration alone does not prove that host-level metrics, application-specific metrics, or notification delivery have been implemented.

14. Testing, Rollback, Troubleshooting, and Final Conclusions

14.1. Introduction

Testing and recovery procedures are essential parts of a reliable DevOps infrastructure. A successful deployment must do more than start the application: it must also be verified, monitored, and recoverable if a problem occurs.

This section describes how to validate the Restaurant Web Application delivery pipeline, test the infrastructure, roll back a failed deployment, and document the final results.

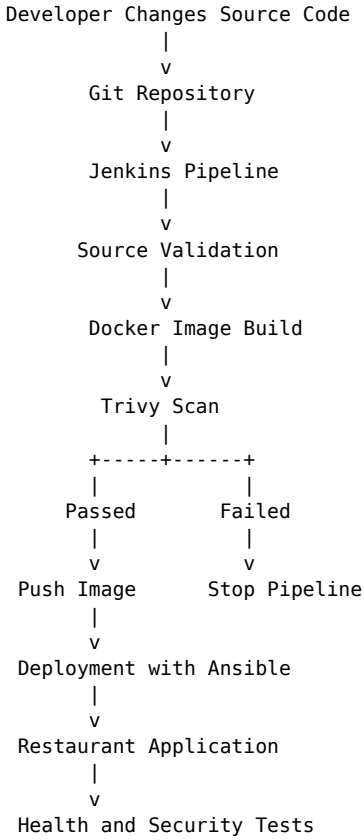
The procedures cover the following areas:

- Git source control and Jenkins pipeline execution.
- Docker image build and deployment.
- Security scanning with Trivy and OWASP ZAP.
- Application availability and load balancing.
- PostgreSQL replication, backup, and recovery.
- Monitoring and alerting.
- Failure simulation and rollback.

Implementation status: Use actual command output, screenshots, scan reports, and test results as evidence. A documented procedure is not proof that a test has passed.

14.2. End-to-End DevOps Workflow

The intended workflow is:



```
        |
        v
Monitoring and Alerting
```

In this workflow, deployment should proceed only after the required validation and security checks succeed.

The actual pipeline may differ depending on which stages have been configured. For example, OWASP ZAP may run after deployment to a test environment rather than before the image is pushed.

14.3. Pre-Deployment Checklist

Before deploying a new application version, verify the following.

Source Code

- Confirm that the correct Git branch and commit are selected.
- Review the changes included in the release.
- Ensure the application files and Dockerfile are present.
- Confirm that the required tests pass.

Docker Image

- Verify that the image builds successfully.
- Use a version-specific tag rather than relying only on `latest`.
- Review the Trivy scan results.
- Confirm that the intended image is available in the registry.

Infrastructure

- Confirm that the target application server is reachable.
- Verify that Docker and Ansible are available where required.
- Confirm that the database is available.
- Check available disk space.
- Confirm that a recent, usable database backup exists when the release could affect data.

Recovery Preparation

- Record the currently deployed image tag or digest.
- Identify the previous known-good version.
- Review any database schema changes.
- Define how to restore the previous version if deployment fails.

14.4. Verify the Application Deployment

After deployment, check the running container:

```
docker ps
```

Inspect the container status:

```
docker inspect --format='{{.State.Status}}' restaurant-app
```

Review recent logs:

```
docker logs --tail=100 restaurant-app
```

Test the application endpoint:

```
curl -I http://127.0.0.1:3001/
```

These commands assume that the container is named `restaurant-app` and the application is exposed on host port `3001`. Replace these values if the actual deployment uses different names or ports.

An HTTP response indicates that the endpoint responded, but it does not necessarily prove that all application functions or database operations work correctly.

If the application provides a dedicated health endpoint, use that endpoint for more meaningful checks.

14.5. Rollback Procedure

Rollback restores a previously working application version after a failed deployment.

The safest approach is to deploy a known-good, versioned image rather than attempting to rebuild an old version from memory.

Step 1 - Identify the Current and Previous Versions

List available images:

```
docker images
```

Record the current deployment version and identify the last known-good version.

Example tags:

```
restaurant-app:25
restaurant-app:26
```

These tags are illustrative. Use the actual image tags from the project.

Step 2 - Verify the Previous Image

Before rollback, confirm that the previous image exists locally or can be pulled from the registry.

```
docker image inspect restaurant-app:25
```

Replace `restaurant-app:25` with the actual known-good image tag.

Step 3 - Deploy the Previous Version

If Ansible manages the deployment, update the image tag in the deployment variables or playbook to the known-good version.

For example:

```
restaurant_image: "restaurant-app:25"
```

This is an example variable; the actual playbook must be configured to use it.

Run the deployment:

```
ansible-playbook -i inventory.ini deploy.yml
```

Use the correct inventory and playbook paths for the environment.

Step 4 - Verify the Rollback

Check the running container:

```
docker ps
```

Inspect the logs:

```
docker logs --tail=100 restaurant-app
```

Test the endpoint:

```
curl -I http://127.0.0.1:3001/
```

Run the relevant application tests and confirm that monitoring reports normal operation.

Important Database Consideration

Rolling back the application image does not automatically reverse database schema changes or data modifications.

Database migrations must be designed for compatibility with both the new and previous application versions where possible. If a migration is destructive or irreversible, a separate recovery plan may be necessary.

Do not restore an older database backup automatically as part of every application rollback. Doing so may discard valid data written after that backup was created.

14.6. Failure Testing

Failure testing verifies how the system behaves when an individual component becomes unavailable.

Perform these tests only in an authorized laboratory environment.

Test	Procedure	Expected result
Application container failure	Stop the application container	The service becomes unavailable or traffic moves to another healthy backend if configured
One backend failure	Stop one application server	HAProxy stops sending traffic to that backend after detecting failure
Load balancer failure	Stop the active Keepalived service	The backup load balancer takes over the VIP if failover is configured correctly
PostgreSQL connection failure	Interrupt database connectivity in the test environment	Application errors are detected and recorded
Jenkins pipeline failure	Introduce a controlled validation error	Jenkins marks the build as failed
Trivy security gate	Test with an image that triggers the configured severity rule	The pipeline stops at the security gate
Backup recovery	Restore a database backup to a test database	The restored database can be queried and validated
Monitoring target failure	Stop a test exporter	Prometheus marks the target as unavailable and the configured alert activates

For each test, record the start time, observed behavior, recovery time, and final result.

A test should be marked as passed only when the expected behavior is observed and supporting evidence is retained.

14.7. Troubleshooting Reference

Application is unavailable

Check:

```
docker ps
docker logs --tail=100 restaurant-app
sudo ss -lntp
curl -v http://127.0.0.1:3001/
```

Verify the container status, port mapping, application logs, and network access.

Jenkins pipeline fails

Check the Jenkins build console output and identify the first failing stage. Confirm that the source repository is accessible and that the Jenkins agent has the necessary Docker, Trivy, and Ansible permissions.

Docker image cannot be pulled

Verify the image name, tag, registry address, credentials, and network connectivity.

Database connection fails

Check PostgreSQL service status, connection settings, credentials, firewall rules, and database logs.

High availability does not work

Verify that both load balancers and application backends are running, HAProxy configuration is valid, and Keepalived can exchange the necessary VRRP traffic.

Monitoring reports missing data

Check Prometheus targets, exporter availability, scrape configuration, network connectivity, and Grafana's data source settings.

Disk space is exhausted

Check:

```
df -h
docker system df
```

Identify which files or containers are consuming storage before deleting anything. Preserve database data, persistent volumes, and required recovery files.

14.8. Evidence Collection

A final DevOps project should include evidence that the infrastructure and procedures have been tested.

Recommended evidence includes:

1. Git repository structure and commit history.
2. Jenkins pipeline configuration and a successful build.
3. Docker image build and registry push output.
4. Trivy scan output and saved report.
5. OWASP ZAP report from the authorized test application.
6. Ansible deployment output.

7. Application response and container status.
8. HAProxy backend health and Keepalived VIP verification, if implemented.
9. PostgreSQL replication status, if implemented.
10. Database backup and successful restoration evidence.
11. Prometheus target status and Grafana dashboards.
12. Rollback and failure-test results.

Screenshots should include enough context to identify the tested component and result. Remove passwords, access tokens, database credentials, private keys, and other sensitive information before including evidence in the report.

14.9. Final Project Assessment

The final assessment should distinguish between completed laboratory work and proposed production capabilities.

Area	Evidence required
Version control	Repository and commit history
CI/CD	Jenkins build logs and pipeline results
Containerization	Docker build and container status
Automation	Ansible execution results
Security	Trivy and ZAP reports
High availability	VIP and failover test results
Database protection	Replication verification and restore test
Monitoring	Prometheus targets and Grafana metrics
Rollback	Successful return to a known-good version

If an item has not been implemented, mark it as planned or not tested. This provides an accurate assessment of the project and identifies future work.

14.10. Limitations and Future Improvements

The following improvements may be required before the design is suitable for production:

- Centralized log collection and retention.
- HTTPS configuration and certificate management.
- Secure management access and secret management.
- Automated database failover with split-brain protection.
- Tested backup scheduling and off-site retention.
- Application-specific health checks.
- Alert notification delivery and response procedures.
- Version-pinned container images and controlled upgrades.
- More comprehensive automated application testing.
- Documented recovery objectives and regular disaster recovery exercises.

The target architecture should be evaluated against actual operational requirements before being used for a production service.

14.11. Conclusion

This project documents a DevOps workflow for the Restaurant Web Application, from source control through automated build, security scanning, deployment, monitoring, and recovery.

Git provides version control, Jenkins coordinates pipeline execution, Docker packages the application, and Ansible supports repeatable deployment. Trivy and OWASP ZAP contribute different types of security testing. HAProxy and Keepalived provide a proposed approach to traffic distribution and load balancer failover. PostgreSQL replication and backups address database availability and data recovery, while Prometheus and Grafana support monitoring and visibility.

The main objective is to make application delivery more repeatable, observable, and recoverable.

The final quality of the project depends not only on the documented architecture but also on the successful execution of its configurations and tests. All implementation claims should be supported by evidence from the actual laboratory environment.

End of Project Documentation